

OE-EC704A — Web Technology

Semester VII • 3L:0T:0P • 3 Credits

Module 10 Exam Notes: Event Handling and AWT

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Delegation Event Model	2
1.1	Common Event Classes & Listeners	2
2	Adapter Classes	2
3	Inner and Anonymous Classes	3
4	AWT Controls	3
5	Layout Managers	3

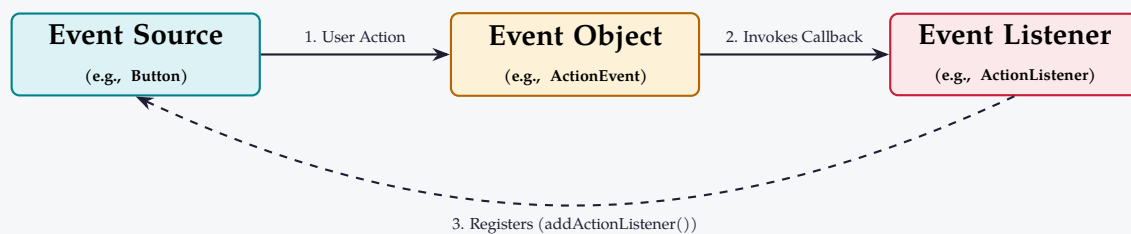
1. Delegation Event Model

Java event handling is built on the **Delegation Event Model**, which separates user interface elements from the logical event processing code.

Core Model Components

- **Event Source:** The GUI component that generates the event when acted upon (e.g., `Button`, `TextField`).
- **Event Object:** An object encapsulating details of the state change (e.g., `ActionEvent`, `MouseEvent`).
- **Event Listener:** An interface defining callback methods to receive and process specific events.

Delegation Event Model Architecture



1.1 Common Event Classes & Listeners

Event Source	Event Class	Listener Interface
Button, TextField	ActionEvent	<code>ActionListener</code>
Component	MouseEvent	<code>MouseListener</code> , <code>MouseMotionListener</code>
Component	KeyEvent	<code>KeyListener</code>
Frame, Window	WindowEvent	<code>WindowListener</code>

2. Adapter Classes

An event listener interface may declare multiple abstract methods. For example, `WindowListener` has seven methods. Implementing the interface directly forces the developer to write empty method bodies for all seven.

Adapter Class Pattern

An **Adapter Class** is a convenience class that implements a listener interface with empty body definitions. By extending the adapter class (e.g., extending `WindowAdapter`), you only need to override the specific methods you require.

3. Inner and Anonymous Classes

To register listeners cleanly without polluting the outer class namespace, Java developers utilize nested class patterns:

- **Inner Classes:** Define a private class inside the AWT frame that implements the listener, accessing the frame's components directly.
- **Anonymous Inner Classes:** Define an inline implementation of the listener/adaptor class without naming it:

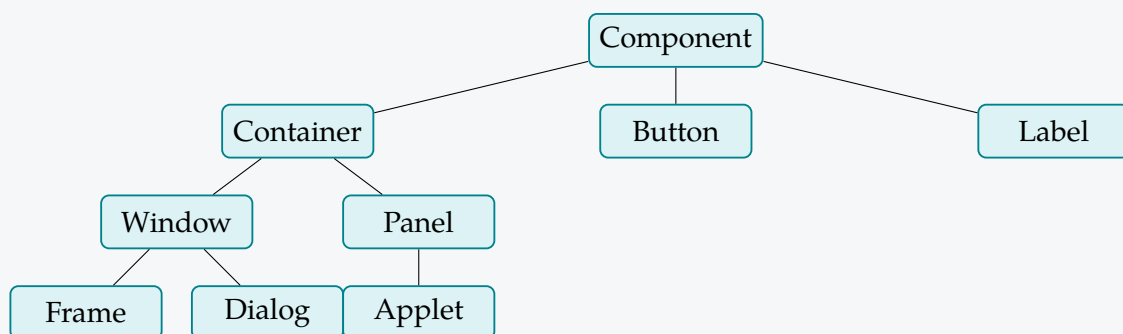
```
btn.addActionListener(new ActionListener() {  
    @Override  
    public void actionPerformed(ActionEvent e) {  
        System.out.println("Button Clicked!");  
    }  
});
```

4. AWT Controls

The Abstract Window Toolkit (AWT) provides platform-dependent heavyweight user interface controls:

- **Label:** Displays a single line of read-only text.
- **Button:** A clickable control that generates action events.
- **TextField / TextArea:** Single-line and multi-line text input fields.
- **Checkbox / Choice:** Selection controls representing boolean states and drop-down lists.

AWT Component Class Hierarchy



5. Layout Managers

Layout managers coordinate the sizing and spatial arrangement of components inside a container:

- **FlowLayout:** Default for `Panel` and `Applet`. Places components in a single line, wrapping left-to-right, top-to-bottom.
- **BorderLayout:** Default for `Frame` and `Window`. Divides space into five geographical regions: `NORTH`, `SOUTH`, `EAST`, `WEST`, and `CENTER`.
- **GridLayout:** Places components in a uniform grid of equal-sized rows and columns.

Quick Revision — Module 10

Event Handling & AWT | OE-EC704A

Event & Layout Syntax

Construct	Syntax and Example Usage
Register Action Listener	<code>btn.addActionListener(listenerObj);</code>
Anonymous Inner Handler	<code>btn.addActionListener(new ActionListener() { ... });</code>
Set Frame Layout	<code>frame.setLayout(new BorderLayout());</code>
Add Component Layout	<code>frame.add(btn, BorderLayout.CENTER);</code>
Window Closing Adapter	<code>f.addWindowListener(new WindowAdapter(){...WindowClosing...});</code>
AWT Frame Init	<code>Frame f = new Frame("Title"); f.setSize(300, 300); f.setVisible(true);</code>

One-Line Definitions

- **Delegation Event Model:** Framework routing UI events from sources to registered listener objects.
- **Event Source:** GUI control generating event objects on user state changes.
- **Event Listener:** Interface declaring abstract event handler callback operations.
- **Adapter Class:** Convenience class providing empty implementations of listener methods.
- **Heavyweight Control:** A UI component bound to and rendered by a native OS peer window.
- **AWT:** Heavyweight platform-dependent Java GUI framework package.
- **Layout Manager:** Object interface deciding component sizing and layouts within containers.
- **Anonymous Inner Class:** Unnamed class declared and instantiated inline at usage point.

Frequently Asked / Viva Questions

Q1. How does the Delegation Event Model work?

An event source (like a button) generates an event object when interacted with, and delegates processing to one or more registered listener objects by invoking their callback methods.

Q2. What is the purpose of Adapter classes in AWT event handling?

They simplify listener implementations. Implementing interfaces like

Frequently Asked / Viva Questions (contd.)

`WindowListener` requires writing empty overrides for all methods. Extending `WindowAdapter` allows overriding only the required methods.

Q3. Why are AWT controls referred to as heavyweight components?

AWT controls are coupled to the underlying operating system. Each AWT component is associated with a native peer window, consuming significant OS resources.

Q4. What is the difference between AWT and Swing?

AWT is platform-dependent and uses heavyweight OS peer rendering. Swing is platform-independent, lightweight, written entirely in Java, and supports a pluggable look-and-feel.

Q5. How does `FlowLayout` arrange components? What is its default alignment?

`FlowLayout` arranges components in a single line from left to right. When it runs out of space, it wraps them to the next line. Its default alignment is centered.

Q6. What layout manager is default for `Frame`? `Panel`?

The default layout manager for a `Frame` is `BorderLayout`. The default layout manager for a `Panel` (and `Applet`) is `FlowLayout`.

Q7. Which listener interface is used to handle button clicks? Text changes?

Button clicks generate `ActionEvents` and are handled by the `ActionListener` interface. Text changes in a `TextField` generate `TextEvents`, handled by the `TextListener` interface.

Q8. How many regions does a `BorderLayout` have? What happens if no region is specified?

Five regions: North, South, East, West, and Center. If no region is specified when adding a component, it is added to the `Center` region by default.

Q9. Why is an anonymous inner class useful in event handling?

It allows declaring and instantiating a listener class inline right where it is registered, keeping code localized and avoiding cluttering the outer class with simple single-method handlers.

Q10. How can you close an AWT `Frame` window when the close button is clicked?

By registering a `WindowListener` (or extending `WindowAdapter`) and calling `System.exit(0)` or `dispose()` inside the overridden `windowClosing(WindowEvent e)` method.

Comparison: AWT vs. Swing

Metric	AWT (Heavyweight)	Swing (Lightweight)
Rendering Engine	OS Native Peer controls.	Painted in Java bytecode.
Platform Styling	Inconsistent look-and-feel.	Uniform style across OS targets.
Class Packages	Lives in <code>java.awt.*</code>	Lives in <code>javax.swing.*</code>
GUI Speed	Slightly faster (OS optimized).	Slightly slower (Java painted).